Tema 3

Identificación de los elementos de un programa informático

Programación

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción 5
2. Estructura y bloques fundamentales 6
3. Identificadores 9
4. Palabras reservadas 10
5. Variables. Declaración, inicialización y utilización. Almacenamiento en memoria 12
5.1 Definición de variable 12
5.2 Cómo declarar una variable en programación 12
5.3 Inicialización y utilización 14
5.4 Almacenamiento en memoria 14
6. Tipos de datos 15
6.1 Tipo de datos para números:int 15
6.2 Double o float son los tipos de datos de números reales 15
6.3 Números complejos 16
6.4 Tipos de datos de cadena: char y strings (str) 17
6.5 Tipos de datos Array (Matriz o Vector) 19
6.6 Datos del tipo de datos booleano 23
6.7 Tipos de datos. Date, Time and Timestamp 24
6.8 Tipo de datos. NoneType 25
7. Literales 26
8. Constantes 27
9. Operadores y expresiones. Precedencia de operadores 28
9.1 Precedencia de Operadores 34
10. Conversiones de tipo. Implícitas y explícitas (casting) 36
10.1 Conversión Implícita 36
10.2 Conversión explícita 37
11. Comentarios 40
11.1 Ejemplos de comentarios 40
12. Resumen 43

Glosario de términos 44
Bibliografía 45

Objetivos de la unidad

Los objetivos de conocimiento de este curso son los siguientes:

- Conocer las partes de un programa (estructura y bloques funcionales).
- Saber lo que es un identificador y cuál es su uso.
- Saber lo que son las palabras reservadas, identificarlas y utilizarlas correctamente.
- Conocer todos los operadores y la precedencia en el uso.
- Adquirir destreza en la conversión de tipos.

Tech Hub <FP> euroformac

1. Introducción

La programación es un proceso creativo. En muchos casos, la tarea en cuestión se puede realizar de muchas maneras diferentes y el programador debe imaginar cuál es la tarea y elegir una de ellas. Algunos de estos métodos pueden ser mejores que otros, pero las computadoras siguen las instrucciones dadas por el programador de todos modos.

Desafortunadamente, las computadoras no entienden español u otros idiomas humanos. Deben recibir instrucciones en un idioma que puedan entender. Para comunicarnos con ellas, necesitamos utilizar lenguajes de programación, que son lenguajes artificiales diseñados para ser comprensibles por las computadoras. Las computadoras interpretan nuestras instrucciones literalmente, por lo que debemos ser muy específicos cuando programamos. Debemos seguir las reglas de su lenguaje de programación y debe eliminar la ambigüedad de las instrucciones dadas.

Python es considerado como un lenguaje de programación muy amigable para principiantes. Tiene una sintaxis clara y legible que facilita la comprensión de los conceptos básicos de la programación.

Una de las ventajas de aprender Python como primer lenguaje de programación es que te ayudará a desarrollar habilidades fundamentales que se aplican en otros lenguajes. Muchos conceptos y principios de programación, como variables, estructuras de control, funciones y estructuras de datos, son comunes a la mayoría de los lenguajes de programación. Al comprender estos conceptos en Python, será más fácil transferir tus conocimientos a otros lenguajes de programación en el futuro.

2. Estructura y bloques fundamentales

En general, la estructura básica de un programa en la mayoría de los lenguajes de programación incluye los siguientes elementos:

- **Declaración de bibliotecas o módulos:** Al comienzo de un programa, es común importar o incluir las bibliotecas o módulos necesarios que proporcionan funcionalidades adicionales. Esto se hace mediante una declaración específica según el lenguaje de programación que estés utilizando.
- **Funciones o procedimientos:** Un programa suele estar compuesto por una serie de funciones o procedimientos. Estas son unidades de código que realizan tareas específicas. Puedes tener una función principal (también llamada "main") que actúa como punto de entrada para el programa y otras funciones que se llaman desde la función principal o entre sí.
- **Declaración de variables:** Las variables se utilizan para almacenar y manipular datos en un programa. Antes de utilizar una variable, generalmente debes declararla, especificando su tipo y nombre. Los tipos de variables pueden variar según el lenguaje de programación, pero algunos tipos comunes incluyen enteros, flotantes, cadenas de texto y booleanos.
- **Entrada y salida de datos:** Los programas a menudo interactúan con los usuarios o acceden a archivos para leer o escribir datos. Por lo tanto, es común incluir instrucciones para recibir datos de entrada (como el teclado) y mostrar resultados o mensajes al usuario (por ejemplo, en la consola o en una interfaz gráfica).

Estos son solo algunos de los bloques fundamentales de un programa, pero hay que tener en cuenta que la estructura y sintaxis específicas pueden variar según el lenguaje de programación que se esté utilizando. Cada lenguaje tiene sus propias convenciones y reglas para la organización del código. Como hemos visto en la unidad 1 un bloque de instrucciones se compone esencialmente de un algoritmo:

Un **algoritmo** tiene estos tres componentes:

- **Entrada (Input):** La entrada en un algoritmo representa los datos o información necesaria para llevar a cabo el proceso. Puede ser proporcionada por el usuario, generada internamente o provenir de fuentes externas, como archivos o sensores. Estos datos de entrada se utilizan como base para realizar las operaciones en el algoritmo.
- **Proceso:** El proceso es la parte central del algoritmo donde se realizan las operaciones y se manipulan los datos de entrada. Aquí es donde se

aplican las instrucciones y cálculos necesarios para transformar los datos de entrada en resultados deseados. El proceso puede incluir operaciones matemáticas, lógicas, comparaciones, estructuras de control, bucles y cualquier otra tarea necesaria para resolver el problema o realizar la tarea específica del algoritmo.

- **Salida (Output):** La salida representa los resultados o la información generada por el algoritmo después de procesar los datos de entrada. Puede ser un valor numérico, un mensaje de texto, una imagen, un archivo generado o cualquier forma de información que sea relevante para la solución del problema. La salida es el resultado final del algoritmo y puede utilizarse como entrada para otros procesos o como resultado visible para el usuario.

En resumen, un algoritmo se inicia con los datos de entrada, pasa por una serie de pasos de procesamiento y finalmente produce una salida como resultado. Estos tres componentes trabajan en conjunto para describir la lógica y la secuencia de operaciones necesarias para resolver un problema o realizar una tarea específica.

De esta manera, un algoritmo informático comienza con un estado inicial y algunos valores de entrada, pasa por una serie de pasos posteriores y finalmente llega a un estado final en el que ha encontrado una solución.

Características de un **algoritmo.** Los algoritmos enumeran una serie de rasgos compartidos, son:

- **Precisos**. Objetivos y bien definidos.
- **Organizados**. Proporcionan un procedimiento claro y detallado para llegar a la respuesta.
- **Finitos**. Tienen un número particular de pasos.
- **Concretos**. Para la situación o problema planteado, presentan una solución detallada.
- **Definidos**. Cuando se le da la misma entrada, el mismo algoritmo debería producir la misma salida.

La siguiente tabla describe la **estructura básica** de un programa informático en bloques.

Cabecera	A modo de comentarios se suele especificar: Nombre del programa. • Nombre del programador. • Fechas de creación y revisiones. • Imput de datos. • Output de datos.
Algoritmos	Funciones propias creadas por el programador para resolver el problema.
Declaraciones	Definiciones y tipos de: Variables. Constantes. Nuevos tipos de datos.
Asignaciones	Valores iniciales de los identificadores declarados previamente.
Entradas	Instrucciones para almacenar en memoria los valores de algunos identificadores.
Control	Instrucciones de control de flujo del programa. Pueden ser: Alternativas. Repetitivas.
Salidas	Instrucciones para devolver los resultados obtenidos.

Estructura básica de un programa informático

3. Identificadores

Una de las novedades introducidas con la aparición de los primeros lenguajes de programación de alto nivel fue la introducción de los denominados **identificadores**.

Definición

Un **identificador** es un nombre definido por el programador que se usa para identificar un elemento particular en un programa. Estos elementos pueden ser variables, constantes y las funciones.

Cuando se inicia un programa el sistema asocia estos nombres con direcciones de memoria. Por lo tanto, cuando programemos, debemos recordar los nombres asignados a estas ubicaciones de memoria, no las ubicaciones de memoria.

En Python, los nombres de variables (identificadores) deben cumplir ciertas condiciones.

- Solo puede contener letras (mayúsculas y minúsculas), números y guiones bajos. Todos los caracteres deben pertenecer a códigos de caracteres ASCII estándar y no deben contener caracteres extendidos.
- El primer carácter no puede ser un número.
- Las **palabras reservadas** no se pueden utilizar como identificadores

4. Palabras reservadas

Definición

Las palabras de programación o palabras clave reservadas tienen un significado especial para el compilador de cualquier lenguaje de programación.

Las **palabras reservadas** en un lenguaje de programación son términos predefinidos que tienen un significado especial y están reservados para realizar tareas específicas. Estas palabras reservadas se utilizan para definir los tipos de datos que se pueden utilizar en el lenguaje y para establecer las diferentes rutinas o estructuras de programación que permiten realizar acciones específicas.

Algunas de las **palabras reservadas comunes** en los lenguajes de programación incluyen:

- **Tipos de datos:** Los lenguajes de programación suelen tener palabras reservadas para definir los tipos de datos, como **int** (entero), **float** (punto flotante), **string** (cadena de caracteres), **bool** (booleano), entre otros. Estas palabras reservadas se utilizan para declarar variables y especificar el tipo de datos que se almacenará en ellas.
- **Estructuras de control:** Los lenguajes de programación tienen palabras reservadas para definir estructuras de control, como **if** (si), **else** (sino), **while** (mientras), **for** (para), **switch** (interruptor), entre otras. Estas palabras reservadas permiten tomar decisiones basadas en condiciones, repetir bloques de código o realizar acciones específicas según la lógica del programa.
- **Funciones y métodos:** Los lenguajes de programación tienen palabras reservadas para definir funciones y métodos, como **def** (definir), **return** (retorno), **void** (vacío), entre otras. Estas palabras reservadas se utilizan para definir bloques de código reutilizables que realizan una tarea específica y pueden ser invocados en diferentes partes del programa.

Es importante tener en cuenta que las palabras reservadas pueden variar según el lenguaje de programación que estemos utilizando. Cada lenguaje tiene su propio conjunto de palabras reservadas con significados y funcionalidades específicas. Es fundamental conocer y comprender las palabras reservadas de un lenguaje de programación para poder utilizarlas

correctamente y aprovechar todas las características y funcionalidades que el lenguaje ofrece. Ejemplo de palabras reservadas en Python:

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Palabras reservadas

5. Variables. Declaración, inicialización y utilización. Almacenamiento en memoria

5.1 Definición de variable

Definición

En programación, una **variable** es un contenedor o espacio de memoria que se utiliza para almacenar y manipular datos. Podemos pensar en una variable como una caja etiquetada donde puedes almacenar diferentes valores, como números, textos, objetos u otros tipos de datos.

Cada variable tiene un nombre único que se utiliza para referirse a ella y acceder a su contenido. El nombre de una variable debe seguir ciertas reglas dependiendo del lenguaje de programación, como no contener espacios en blanco y comenzar con una letra o un carácter especial permitido. En términos simples, si desea describir qué es una variable en la programación, puede decir que es una unidad de almacenamiento y recuperación de datos que puede cambiar un valor identificado por un nombre único en el código del programa. Además, en programación, una variable puede entenderse como un nombre que representa un valor y se usa periódicamente para almacenar varios tipos de datos que se usan en el trabajo de codificación posterior.

5.2 Cómo declarar una variable en programación

En cualquier lenguaje de programación, podemos declarar distintos tipos de variables. Podemos representar los siguientes tipos de datos:

- **Variables simples numéricas, entre estas:**
  - *Enteros*: Los enteros son valores numéricos sin parte decimal. Pueden ser positivos o negativos, y no contienen fracciones ni decimales. Los enteros se utilizan para representar cantidades enteras, como la cantidad de elementos en una lista o el número de repeticiones de un bucle.
  - *Reales*. Los números reales son valores que pueden contener una parte decimal. Se utilizan para representar cantidades fraccionarias o decimales. Los números reales son útiles cuando se requiere precisión decimal en cálculos matemáticos o cuando se necesitan valores con decimales en general.

- **Variables múltiples numéricas, entre estas:**
  - *Vectores*. Contiene un vector de valores.
  - *Matrices*. Contiene una matriz de valores.

- **Cadenas de caracteres**. Contiene una cadena de caracteres.
- **Registros**.

La asignación de valores a las variables se lleva a cabo mediante la provisión de un identificador para cada una, seguido de la utilización del signo igual para establecer su valor, siguiendo el siguiente formato:

Enteros:

```
ent = 2; % Este crea una variable llamada 'ent' con el valor 2
```

Reales:

```
real = 1.5; % Este crea una variable llamada 'real' con el valor 1,5
```

Cadenas de caracteres, un tipo de cadena sería:

```
cadena = 'Hola mundo'; % Este crea una variable llamada 'cadena' con la cadena de caracteres 'Hola mundo'.
```

El uso del punto y coma (;) al final de una declaración no afecta directamente si una variable se muestra o no en el terminal. La visualización de una variable en el terminal o en la salida del programa depende de cómo se haya implementado el código para imprimir o mostrar la variable específica. Para asignar un nuevo valor a una variable, debe, como antes, declarar el nuevo valor que utilizará la variable.

Por ejemplo:

```
ent = 2;

ent = 3;
```

El valor de **ent** ha pasado de ser 2 a ser 3.

Cuando a una variable no se le da un identificador, es decir, se declara únicamente su valor, se asigna el identificador **ans**.

Para visualizar una variable tan solo tendremos que introducir su **identificador**.

5.3 Inicialización y utilización

Muchas veces es necesario inicializar una variable antes de utilizarla. La inicialización de una variable simplemente consiste en asignarle el valor predeterminado que necesitamos que esta tenga.

Por ejemplo, si se va a utilizar una variable para crear un contador, sería una buena idea asignar (inicializar) el valor 0 (cero) a la misma.

```
Contador = 0;
```

5.4 Almacenamiento en memoria

En la memoria de una computadora, los datos se almacenan en celdas o posiciones de memoria, y cada una de ellas tiene una dirección única. Los programas de computadora utilizan identificadores (también conocidos como variables) para referirse a estas ubicaciones de memoria y manipular los datos almacenados en ellas.

Los identificadores son nombres que se asignan a las variables para identificarlas y acceder a ellas en el código. Los identificadores pueden ser palabras descriptivas que representen el propósito o el contenido de la variable. Al utilizar identificadores, los programadores no necesitan preocuparse por las direcciones de memoria específicas; en su lugar, pueden trabajar con los identificadores y dejar que el lenguaje de programación se encargue de la asignación y administración de la memoria.

Los lenguajes de programación de alto nivel, como Python, Java o C++, proporcionan una sintaxis específica para declarar e utilizar identificadores. Al declarar una variable con un identificador, se reserva un espacio de memoria y se asocia el nombre del identificador con esa ubicación. Luego, se puede utilizar el identificador para almacenar datos en esa ubicación y recuperarlos más tarde.

6. Tipos de datos

Cada lenguaje de programación es capaz de manejar una amplia variedad de datos. Sin embargo, siempre tendremos acceso a los tipos de datos básicos.

Estos están incluidos en el lenguaje de programación, lo que nos permite realizar tareas más difíciles.

Repasemos los tipos de datos ya vistos en la Unidad 1 en Python por ser un lenguaje muy sencillo de comprender.

6.1 Tipo de datos para números:int

Dado que es importante resolver una serie de problemas, los números generalmente se representan en un lenguaje de programación de varias maneras:

- ¿Qué **tamaño** emplearemos?
- Si los números estarán en **forma decimal**.
- Si el resultado va a ser **negativo**.

Comenzaremos con el tipo primitivo **int**. Cualquier número, positivo o negativo, sin decimales se representa con este tipo de datos.

Hay algunos lenguajes, como **Pascal**, que lo escriben como **integer**, aunque con frecuencia se escribe como **int** en el código fuente.

```
edad = 30
type(edad)  # Comando que nos indica el tipo de datos <class 'int'> #resultado de la salida de comando Type.
```

6.2 Double o float son los tipos de datos de números reales

Los tipos de datos **double** o **float** se utilizan normalmente cuando se trabaja con números decimales.

Este es un número de "coma flotante", como decimos. Según el nivel de precisión decimal que desees, tendrás que declararlo de un tipo u otro.

El tipo **double** tiene un rango de +/- 1.8 * 10^-308 a +/- 1.8 * 10^308.

El tipo **float** tiene un rango de +/- 3.4 * 10^-38 a +/- 3.4 * 10^38.

Otra forma de saber si un número es flotante o doble es verlo con la letra F o la letra D.

Por ejemplo 4.89F.

```
altura=1.85
type(altura) #Comando que nos indica el tipo de datos
<class 'float'>
```

6.3 Números complejos

Definición

En Python, los números complejos se pueden representar utilizando la función `complex () `. Un número complejo consta de una parte real y una parte imaginaria, donde la parte imaginaria se denota con la unidad imaginaria `j` o `J`.

Aquí tenemos un ejemplo de cómo se define un número complejo en Python:

```
numero_complejo = complex(3, 4)
print(numero_complejo)
```

En este caso, el número complejo se define con una parte real de 3 y una parte imaginaria de 4. Al imprimir `numero_complejo`, obtendremos el resultado:

```
(3+4j)
```

Podemos acceder a la parte real e imaginaria de un número complejo utilizando los atributos `real` e `imag`, respectivamente. Aquí tienes un ejemplo:

```
parte_real = numero_complejo.real
parte_imaginaria = numero_complejo.imag
print(parte_real)        # Imprime 3.0
print(parte_imaginaria)    # Imprime 4.0
```

Además, podemos realizar operaciones matemáticas con números complejos en Python, como suma, resta, multiplicación y división. Python proporciona

funciones y operadores para trabajar con números complejos. Aquí tenemos algunos ejemplos:

```
# Suma de números complejos
resultado_suma = complex(2, 3) + complex(1, 2)
print(resultado_suma)     # Imprime (3+5j)

# Resta de números complejos
resultado_resta = complex(5, 7) - complex(2, 3)
print(resultado_resta)     # Imprime (3+4j)

# Multiplicación de números complejos
resultado_mult = complex(2, 1) * complex(3, 4)
print(resultado_mult)     # Imprime (-5+10j)

# División de números complejos
resultado_div = complex(4, 5) / complex(2, 1)
print(resultado_div)     # Imprime (2.6+0.8j)
```

6.4 Tipos de datos de cadena: char y strings (str)

En Python, no existe un tipo de datos de "char" como en algunos otros lenguajes de programación. En su lugar, el tipo de datos de cadena en Python se representa con el tipo `str`, que se utiliza para almacenar secuencias de caracteres.

Aquí tenemos una breve definición de los tipos de datos de cadena en Python:

Definición

`str`: El tipo de datos `str` representa una cadena de caracteres. Puedes crear una cadena en Python utilizando comillas simples ('') o comillas dobles ("").

Por ejemplo:

```
cadena1 = 'Hola, mundo!'
cadena2 = "¡Hola, Python!"
```

Las cadenas en Python son inmutables, lo que significa que no podemos modificar los caracteres individuales de una cadena después de haberla

creado. Sin embargo, puedes realizar diversas operaciones y manipulaciones de cadenas, como concatenación, extracción de subcadenas, búsqueda y reemplazo de caracteres, entre otros.

```
# Concatenación de cadenas
cadena_concatenada = cadena1 + ' ' + cadena2
print(cadena_concatenada)    # Imprime "Hola, mundo! ¡Hola,
Python!"

# Extracción de subcadenas
subcadena = cadena1[0:5]
print(subcadena)             # Imprime "Hola,"

# Búsqueda y reemplazo de caracteres
cadena_modificada = cadena1.replace('mundo', 'Python')
print(cadena_modificada)     # Imprime "Hola, Python!"
```

Además, Python proporciona una amplia variedad de métodos y funciones incorporadas para trabajar con cadenas, como `len()`, `lower()`, `upper()`, `split()`, entre otros, que nos permiten realizar diferentes operaciones y manipulaciones de cadenas.

Es importante tener en cuenta que las cadenas en Python son secuencias de caracteres Unicode, lo que significa que podemos trabajar con caracteres de diferentes idiomas y símbolos especiales sin problemas. Esto brinda una gran flexibilidad en el manejo de cadenas en Python.

Importante

Normalmente, escribimos el carácter Unicode conocido como "char" entre comillas simples ('').

Sin embargo, solemos identificar un carácter como una cadena si es una cadena de caracteres, que son caracteres que se siguen uno tras otro para formar una secuencia.

Para distinguirlo de char, el tipo de string (str) debe estar entre comillas dobles (“ “), aunque puede variar según el lenguaje de programación.

En resumen, en Python, utilizamos el tipo de datos `str` para representar cadenas de caracteres, y no hay un tipo de datos específico para un solo carácter como "char" en otros lenguajes. Las cadenas en Python son

inmutables y proporcionan numerosas operaciones y funciones para manipular y trabajar con ellas.

6.5 Tipos de datos Array (Matriz o Vector)

Definición

El término "array" se utiliza ampliamente en la programación para referirse a una estructura de datos que almacena elementos del mismo tipo de manera contigua en la memoria.

Los **arrays** permiten acceder a los elementos mediante un índice numérico y ofrecen eficiencia en términos de acceso y manipulación de datos.

A continuación, presentamos algunos tipos de datos "array" que se utilizan en diferentes lenguajes de programación:

- **Array unidimensional:** También conocido como vector, es una colección de elementos del mismo tipo dispuestos en una sola dimensión. Los elementos se acceden mediante un índice. Ejemplos de lenguajes que admiten arrays unidimensionales son C, C++, Java y Python.
- **Array bidimensional:** También conocido como matriz, es una estructura de datos con elementos organizados en filas y columnas. Se utiliza para representar tablas, matrices y estructuras similares. Los elementos se acceden utilizando dos índices: uno para la fila y otro para la columna. Lenguajes como C, C++, Java y Python admiten arrays bidimensionales.
- **Array multidimensional:** Es una extensión del array bidimensional y permite almacenar elementos en más de dos dimensiones. Por ejemplo, un array tridimensional se utiliza para representar cubos o volúmenes. Lenguajes como C, C++, Java y Python admiten arrays multidimensionales.

Importante

Es importante tener en cuenta que la implementación y características de los arrays pueden variar según el lenguaje de programación. Algunos lenguajes proporcionan funcionalidades adicionales y métodos específicos para manipular arrays, como la capacidad de ordenar, buscar o realizar operaciones matemáticas en los elementos del array.

En resumen, los tipos de datos "array" se utilizan en la programación para almacenar elementos del mismo tipo de manera contigua en la memoria. Los arrays unidimensionales, bidimensionales y multidimensionales son algunos ejemplos comunes de arrays utilizados en diferentes lenguajes de programación.

A continuación, mostramos un ejemplo sencillo de un array en Python es el siguiente:

```
# ingredientes declarados como variables independientes
ingrediente1 = "leche"
ingrediente2 = "harina"
ingrediente3 = "azucar"
# Ingredientes añadidos en la lista
ingredientes = [ingrediente1, ingrediente2, ingrediente3]
```

En este caso, tendríamos una lista de ingredientes, a cada uno de los cuales le asignamos un valor fijo (ingrediente1, 2, 3, etc.). Sin embargo, si la lista es demasiado larga, se puede filtrar usando una matriz de Python para encontrar un elemento específico. Podemos agrupar tantos valores como quieras seleccionándolos por el número que tienen asignado.

- **Acceder a un elemento específico**

Para acceder a un elemento concreto, utiliza el número de índice. Este es un ejemplo en el que primero se consulta el valor del primer elemento:

```
ingredientes[0]
```

'leche'

En el segundo paso, se introduce un nuevo valor en el array:

```
ingredientes[0]="crema"
ingredientes[0]
```

'crema'

- **La longitud de un array en Python**

La longitud de un array en Python debe determinarse de antemano y no puede modificarse más adelante. Para establecer la longitud, seleccionaremos el valor más alto de los números de índice previstos y lo aumentamos en 1. Para obtener la longitud del array en Python, usaremos la función **"len ( )"**. Aquí tenemos un ejemplo:

```
numero_ingredientes = len(ingredientes)
numero_ingredientes
3
```

- **Añadir elementos a un array en Python**

Para añadir elementos a nuestro array en Python, lo mejor es utilizar el método " append ( )" . Este funciona de la siguiente manera:

```
ingredientes.append("sal")
ingredients[3] #tenemos en cuenta que la posición del
primer elemento es la cero.
'sal'
ingredientes

['crema', 'harina', 'azucar', 'sal']
```

- **Eliminar elementos del array con pop ( ) o remove ( )**

Existen dos formas de eliminar elementos de un array en Python. El primer método es **"pop ( )"**. Si quieres eliminar el tercer ingrediente (" sugar") del conjunto de ingredientes introducido anteriormente, utiliza el siguiente código:

```
tercero = ingredientes.pop(2)
tercero
'azucar'
ingredientes
['crema', 'harina', 'sal'] # valores que restan en el array
tras la eliminación del elemento "sugar".
```

Hay que tener en cuenta que se empieza a contar desde 0, por lo que el primer elemento toma el valor 0, el segundo el valor 1, y así sucesivamente.

El segundo método para eliminar el tercer ingrediente del array en Python es **"remove ( )"**. Este tiene el siguiente aspecto:

```
ingredientes.remove("crema")
ingredientes

['harina', 'sal']
```

De esta manera, el valor " crema" se extrae del array. Sin embargo, es importante saber que este método solo elimina el valor correspondiente la

primera vez que aparece. Si el valor vuelve a aparecer de nuevo en el transcurso de la lista, no será eliminado de forma automática.

- **Crear un bucle de elementos en un array**

En Python podemos crear un bucle de elementos del array con un bucle " for in". Con el siguiente comando se solicita asignar un ingrediente cada elemento del array " ingredientes" de Python:

```
for ingrediente in ingredientes:
    print (ingrediente)
#salida por pantalla
harina
sal
```

- **Lista de diferentes métodos para arrays en Python**

Para un procesamiento más rápido y específico de estos arrays, deberíamos echar un vistazo a los diferentes métodos proporcionados por Python. Los siguientes métodos nos ayudarán a aprender más lenguajes de programación y trabajar con arreglos en Python:

Método	Descripción
append	Añade un elemento al final de la lista
clear	Elimina todos los elementos de la lista
copy	Crea una copia de toda la lista
count	Da como resultado el número exacto de elementos con un valor determinado
extend	Añade todos los elementos de una lista al final de un array en Python
index	Muestra el número de índice del primer elemento con un valor determinado
insert	Añade un elemento en una posición determinada
len	Se determina la longitud de un array en Python
pop	Se elimina un elemento en una posición determinada

remove	Elimina el primer elemento con un valor determinado
reverse	Invierte el orden de los elementos del array
sort	Ordenar una lista

6.6 Datos del tipo de datos booleano

Definición

El tipo primitivo **boolean** representa valores booleanos. Muestra si una condición es verdadera o falsa.

Por lo general, solo tienen identificados los valores **true** (verdadero) o **false** (falso). Puede ser lo mismo que los números 0 y 1 en algunos idiomas.

Dos valores cualesquiera, o estados, pueden representarse mediante una variable.

En programación, los operadores lógicos se utilizan para realizar operaciones de comparación y evaluar condiciones booleanas. Los operadores lógicos más comunes son AND, OR, XOR y NOT. Aquí tienes una explicación de cada uno de ellos:

- **AND (Y lógico):** El operador AND se representa con el símbolo "&&" en muchos lenguajes de programación. Este operador devuelve true si ambos operandos son verdaderos, y false en cualquier otro caso. La tabla de verdad para el operador AND es la siguiente:

Operando 1	Operando 2	Resultado
true	true	true
true	false	false
false	true	false
false	false	false

- **OR (O lógico):** El operador OR se representa con el símbolo "||" en muchos lenguajes de programación. Este operador devuelve true si al menos uno de los operandos es verdadero, y false si ambos operandos son falsos. La tabla de verdad para el operador OR es la siguiente:

Operando 1	Operando 2	Resultado
true	true	true
true	false	true
false	true	true
false	false	false

- **XOR (O exclusivo):** El operador XOR se representa con el símbolo "^" en muchos lenguajes de programación. Este operador devuelve true si exactamente uno de los operandos es verdadero, y false en cualquier otro caso. La tabla de verdad para el operador XOR es la siguiente:

Operando 1	Operando 2	Resultado
true	true	true
true	false	true
false	true	true
false	false	false

- **NOT (Negación):** El operador NOT se representa con el símbolo "!" en muchos lenguajes de programación. Este operador invierte el valor de un operando booleano. Si el operando es true, el operador NOT devuelve false, y si el operando es false, devuelve true.

Estos operadores lógicos se utilizan para construir expresiones booleanas y evaluar condiciones en estructuras de control como las declaraciones if-else, bucles y expresiones condicionales. Permiten realizar comparaciones y tomar decisiones basadas en los valores booleanos de las variables o expresiones.

6.7 Tipos de datos. Date, Time and Timestamp

Estos tipos de datos se utilizan para trabajar con datos que contienen fechas y horas.

Tipo de dato	Valor
DATE	Año, mes y día
TIME	Hora, minuto y segundo
TIMESTAMP	Año, mes, día, hora, minuto, segundo y microsegundo

6.8 Tipo de datos. NoneType

Cuando ningún valor es válido o para indicar que una variable aún no tiene un valor significativo, el valor **None** se utiliza para representar estas situaciones.

El valor **None** tiene un tipo único llamado **NoneType** que lo distingue de todos los demás valores.

7. Literales

Definición

Un literal es un dato específico escrito como en un programa que es un valor literal, como el número -1634.35 o la palabra casa.

Existen algunas reglas que debemos recordar:

- Los números enteros se escriben sin punto de millar porque los números reales usan puntos para separar los puntos decimales de los números enteros.
- Los números reales se escriben con decimales en lugar de comas para separar decimales.
- Encierre los caracteres entre comillas simples (`a´) para que no se confundan con variables.
- Las cadenas se encierran entre comillas dobles ("hola") para evitar confusiones con las variables.
- Los valores booleanos se escriben como verdadero/falso o 1/0.

Sabías que

Los literales se utilizan en el código para proporcionar valores específicos a las variables, realizar operaciones matemáticas, comparar valores y más. Son constantes que no cambian su valor a lo largo de la ejecución del programa.

8. Constantes

Definición

Estamos hablando de constantes para un tipo especial de variable cuyo valor es inmutable (es incorrecto llamarlo variable constante).

Inmutable significa que no se puede cambiar durante la ejecución del programa, es decir, durante la ejecución. Esta es la diferencia clave entre constante y variable. Las variables pueden contener cualquier valor (similar a los tipos de datos que declaramos), ya sea modificando en el código fuente en el momento del diseño o en el momento de la ejecución del programa.

Todos conocemos las matemáticas, donde tenemos las dos constantes más famosas que son:

- El número Pi π.
- El número e.

Importante

Es importante tener en cuenta que, si bien las constantes no se pueden modificar, algunos lenguajes de programación permiten declarar constantes en tiempo de compilación que pueden ser evaluadas y asignadas en tiempo de ejecución. Además, las convenciones de nomenclatura de las constantes pueden variar, pero suelen usar letras mayúsculas y separadores de palabras, como guiones bajos o camel case, para mejorar la legibilidad.

9. Operadores y expresiones. Precedencia de operadores

En programación, los operadores se utilizan para realizar operaciones en expresiones, que son combinaciones de valores, variables y operadores. Los operadores permiten realizar cálculos matemáticos, comparaciones, asignaciones y otras operaciones en los datos.

Los operadores pueden clasificarse en diferentes categorías según el tipo de operación que realizan, como operadores aritméticos, operadores de asignación, operadores de comparación, operadores lógicos, etc.

Al trabajar con expresiones que contienen múltiples operadores, es importante tener en cuenta la precedencia de los operadores. La precedencia determina el orden en que se evalúan los operadores en una expresión. Cuando hay múltiples operadores en una expresión, los operadores con mayor precedencia se evalúan antes que los operadores con menor precedencia. Estos son los diferentes tipos de operadores en Python:

- **Operador aritmético**

Los operadores aritméticos toman dos operandos como entrada, realizan cálculos y devuelven un resultado.

Considere la expresión "a = 2 + 3". Donde 2 y 3 son operandos y operadores aritméticos. El resultado final de la operación realizada se almacena en la variable a.

OPERADOR	DESCRIPCIÓN	USO
+	Realiza Adición entre los operandos	12 + 3 = 15
-	Realiza Substracción entre los operandos	12 - 3 = 9
*	Realiza Multiplicación entre los operandos	12 * 3 = 36
/	Realiza División entre los operandos	12 / 3 = 4
%	Realiza un módulo entre los operandos	16 % 3 = 1
**	Realiza la potencia de los operandos	12 ** 3 = 1728
//	Realiza la división con resultado de número entero	18 // 5 = 3

Operadores Aritméticos

- **Operadores relacionales**

Los operadores relacionales se utilizan para comparar y establecer relaciones entre ellos. Devuelve un valor booleano (verdadero o falso) según la condición.

OPERADOR	DESCRIPCIÓN	USO
>	Devuelve True si el operador de la izquierda es mayor que el operador de la derecha	12 > 3 devuelve True
<	Devuelve True si el operador de la derecha es mayor que el operador de la izquierda	12 < 3 devuelve False
==	Devuelve True si ambos operandos son iguales	12 == 3 devuelve False
>=	Devuelve True si el operador de la izquierda es mayor o igual que el operador de la derecha	12 >= 3 devuelve True
<=	Devuelve True si el operador de la derecha es mayor o igual que el operador de la izquierda	12 <= 3 devuelve False
!=	Devuelve True si ambos operandos no son iguales	12 != 3 devuelve True

operadores relacionales

- **Operadores bit a bit**

Los operadores bit a bit realizan operaciones utilizando operandos bit a bit.

Consideremos a = 2 (binario = 10) y b = 3 (binario = 11) para los siguientes casos:

OPERADOR	DESCRIPCIÓN	USO
&	Realiza bit a bit la operación AND en los operandos	a & b = 2 (Binario: 10 & 11 = 10)
|	Realiza bit a bit la operación OR en los operandos	a | b = 3 (Binario: 10 | 11 = 11)

OPERADOR	DESCRIPCIÓN	USO
^	Realiza la operación XOR bit a bit en los operandos	a ^ b = 1 (Binario: 10 ^ 11 = 01)
~	Realiza la operación NOT bit a bit. Invierte cada bit en el operando	~a = -3 (Binario: ~(00000010) = (11111101))
>>	Realiza un desplazamiento a la derecha bit a bit. Desplaza los bits del operador de la izquierda a la derecha tantos bits como indica el operador de la derecha	a >> b = 0 (Binario: 00000010 >> 00000011 = 0)
<<	Realiza un desplazamiento a la izquierda bit a bit. Desplaza los bits del operando de la izquierda a la izquierda tantos bits como especifique el operador de la derecha	a << b = 16 (Binario: 00000010 << 00000011 = 00001000)

Operadores bit a bit

- **Operadores de asignación**

Los operadores de asignación se utilizan para asignar o inicializar valores a las variables. Por lo general, se combina con otros operadores (p. ej., aritmético, bit a bit) donde se realiza una operación en un operando y el resultado se asigna al operando izquierdo.

Consideremos los siguientes ejemplos.

a = 18. Aquí el **operador =** es un operador de asignación, y el resultado se almacena en la variable a.

a + = 10. Aquí += es un operador de asignación, y el resultado se almacena en la variable a. Es lo mismo que a = a + 10.

OPERADOR	DESCRIPCIÓN
=	a = 5. El valor 5 es asignado a la variable a
+=	a += 5 es equivalente a a = a + 5
-=	a -= 5 es equivalente a a = a - 5
*=	a *= 3 es equivalente a a = a * 3

OPERADOR	DESCRIPCIÓN
/=	a /= 3 es equivalente a a = a / 3
%=	a %= 3 es equivalente a a = a % 3
**=	a **= 3 es equivalente a a = a ** 3
//=	a //= 3 es equivalente a a = a // 3
&=	a &= 3 es equivalente a a = a & 3
|=	a |= 3 es equivalente a a = a | 3
^=	a ^= 3 es equivalente a a = a ^ 3
>>=	a >>= 3 es equivalente a a = a >> 3
<<=	a <<= 3 es equivalente a a = a << 3

Operadores de asignación

- **Operadores lógicos**

Los operadores lógicos desempeñan un papel crucial en la toma de decisiones fundamentadas en diversas condiciones. En el ámbito de Python, los operadores lógicos más frecuentemente empleados son **and**, **or** y **not**.

OPERADOR	DESCRIPCIÓN	USO
and	Devuelve True si ambos operandos son True	a and b
or	Devuelve True si alguno de los operandos es True	a or b
not	Devuelve True si alguno de los operandos False	not a

Operadores lógicos

- **Operadores de Pertenencia**

Los operadores **in** y **not** son operadores utilizados en algunos lenguajes de programación para realizar operaciones de pertenencia y negación, respectivamente.

- **Operador in:** El operador in se utiliza para verificar si un valor está presente en una secuencia, como una lista, una cadena de texto o una tupla. Devuelve True si el valor está presente y False en caso contrario. En términos de precedencia,

el operador in generalmente tiene una precedencia más baja que los operadores aritméticos y de comparación.

- **Operador not:** El operador not se utiliza para negar una expresión booleana. Si una expresión es verdadera, el operador not la convierte en falsa, y si es falsa, la convierte en verdadera. En términos de precedencia, el operador not generalmente tiene una precedencia más alta que los operadores de comparación y lógicos.

Importante

Es importante tener en cuenta que la precedencia de los operadores puede variar según el lenguaje de programación. Por lo tanto, siempre es recomendable consultar la documentación específica del lenguaje que estés utilizando para conocer la precedencia exacta de los operadores. Además, puedes utilizar paréntesis para cambiar el orden de evaluación y asegurarte de que las expresiones se evalúen correctamente según tus necesidades.

```
a = [1, 2, 3, 4, 5]

# Esta 3 en la lista a?
print
3 in a  # Muestra True

# No está 12 en la lista a?
print
12 not in a  # Muestra True

str = "Hello World"

# Contiene World el string str?
print
"World" in str  # Muestra True

# Contiene world el string str? (nota: es casesensitive , es decir  distingue mayúsculas y minúsculas)
print
"world" in str  # Muestra False (0)

print
"code" not in str  # Muestra True (1)
```

- **Operadores de Identidad**

Los operadores de identidad, is y is not, se utilizan en algunos lenguajes de programación para comparar la identidad de dos objetos. A diferencia de los operadores de igualdad (== y !=), que comparan los valores de los objetos, los operadores de identidad verifican si dos objetos son el mismo objeto en memoria.

- **Operador is:** El operador is verifica si dos objetos son el mismo objeto en memoria. Devuelve True si los objetos son idénticos y False en caso contrario.

- **Operador is not:** El operador is not es la negación del operador is. Devuelve True si los objetos no son el mismo objeto en memoria y False si son idénticos.

Importante

Es importante destacar que los operadores de identidad se utilizan para comparar objetos, no para comparar los valores contenidos en ellos. En el caso de objetos inmutables como números y cadenas de texto, los operadores de identidad pueden dar los mismos resultados que los operadores de igualdad (== y !=). Sin embargo, para objetos mutables como listas o diccionarios, los operadores de identidad pueden ser útiles para verificar si dos variables se refieren al mismo objeto en memoria.

La precedencia de los operadores de identidad puede variar según el lenguaje de programación, pero generalmente tienen una precedencia similar a los operadores de comparación. Es recomendable consultar la documentación específica del lenguaje que estés utilizando para conocer la precedencia exacta de los operadores.
Tenemos en cuenta que dos valores, cuando son iguales, no implican necesariamente que sean idénticos.

```
a = 3
b = 3
c = 4
print a is b # muestra True
print a is not b # muestra False
print a is not c # muestra True

x = 1
y = x
z = y
print z is 1 # muestra True
print z is x # muestra True
```

```
str1 = "FreeCodeCamp"
str2 = "FreeCodeCamp"

print str1 is str2 # muestra True
print "Code" is str2 # muestra False

a = [10,20,30]
b = [10,20,30]

print a is b # muestra False (ya que las listas son objetos mutables en Python)
```

9.1 Precedencia de Operadores

La precedencia de operadores determina el orden en que los operadores son evaluados dentro de una expresión. Cuando una expresión contiene múltiples operadores, la precedencia define qué operadores se evalúan primero y cuáles se evalúan después.

Aquí tienes una lista general de la **precedencia de operadores**, desde la mayor a la menor:

- **Paréntesis**: Los paréntesis ( ) se utilizan para agrupar subexpresiones y tienen la mayor precedencia. Lo que está dentro de los paréntesis se evalúa primero.
- **Exponenciación**: El operador de exponenciación ** tiene la siguiente precedencia más alta. Se utiliza para calcular una potencia.
- **Multiplicación, división y módulo:** Los operadores de multiplicación *, división / y módulo % tienen la misma precedencia y se evalúan de izquierda a derecha.
- **Suma y resta:** Los operadores de suma + y resta - tienen la misma precedencia y se evalúan de izquierda a derecha.
- **Operadores de comparación:** Los operadores de comparación, como igual ==, distinto! =, mayor que >, menor que <, mayor o igual que >= y menor o igual que <=, tienen una precedencia menor que los operadores aritméticos.
- **Operadores lógicos:** Los operadores lógicos, como AND &&, OR || y NOT !, tienen una precedencia menor que los operadores aritméticos y de comparación.

Es importante tener en cuenta que la precedencia de los operadores puede variar según el lenguaje de programación. Algunos lenguajes también proporcionan reglas adicionales para la resolución de conflictos de precedencia en casos específicos. En caso de duda, es recomendable consultar la documentación del lenguaje de programación específico que estés utilizando para conocer la precedencia exacta de los operadores. Además, puedes utilizar paréntesis para cambiar el orden de evaluación y asegurarte de que las expresiones se evalúen según tus necesidades.

OPERADOR	DESCRIPCIÓN
(expresiones...) [expresiones...], {clave: valor...} {expresiones...}	Visualización o unión de tuplas, listas, diccionarios o conjuntos
x[índice], x[índice:índice], x(argumentos...), x.atributo	Extracción de datos de estructuras, referencia a atributos
await x	Expresión await
**	Exponenciación
+x, -x, ~x	Especificación de signo, operador "no" a nivel de bits
*, @, /, //, %	Multiplicación, multiplicación de matrices, división, "floor division", resto
+, -	Adición y sustracción
<<, >>	Desplazamiento a nivel de bits
&	"y" a nivel de bits
^	"xor" a nivel de bits
|	"o" a nivel de bits
in, not in, is, is not, <, <=, >, >=, ==, !=	Comparaciones, identificación y pertenencia
not x	"no" booleano
and	"y" booleano
or	"o" booleano
if - else	Expresión condicional
lambda	Expresión lambda

Precedencia de los operadores

10. Conversiones de tipo. Implícitas y explícitas (casting)

Realizar un cast o casting implica la conversión de un tipo de dato a otro. A continuación, examinaremos las diversas modalidades de cast o conversión de tipos disponibles en el lenguaje de programación Python.

Existen dos tipos:

- **Conversión implícita:** se refiere a la transformación automática de un tipo de dato a otro tipo compatible sin necesidad de una intervención explícita por parte del programador. Es decir, cuando se realiza una operación o asignación entre variables de diferentes tipos, el lenguaje de programación realiza la conversión automáticamente para que la operación sea válida y pueda ser ejecutada.
- **"Conversión explícita:** también conocida como "casting" o "type casting", se refiere a la acción de cambiar o convertir el tipo de datos de una variable de forma deliberada y explícita. Esto implica indicar de manera específica la conversión deseada mediante la utilización de operadores o funciones proporcionadas por el lenguaje de programación.

10.1 Conversión Implícita

Es importante saber lo que pasa por debajo para evitar problemas futuros. El ejemplo más sencillo donde podemos ver este comportamiento es el siguiente:

```
a = 1   # <class 'int'>
b = 2.3 # <class 'float'>

a = a + b
print(a)      # 3.3
print(type(a)) # <class 'float'>
```

```
a es un int

b es un float
```

Cuando se suman un entero (int) y un número en punto flotante (float) en Python, el resultado se promocionará a un número de punto flotante. En resumen, Python realiza conversiones automáticas en algunos casos, pero no

todos los tipos de datos son compatibles con todas las operaciones. En los casos en los que no se pueda realizar una conversión automática, es necesario realizar la conversión manualmente antes de realizar la operación deseada.

```
a = 1
b = "2.3"

c = a + b

# TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

10.2 Conversión explícita

Es posible hacer, de forma explícita, cast (o conversiones entre tipos) mediante diversas funciones de Python. Las más usadas son las siguientes:

- float(), str(), int(), list(), set()
- Y algunas otras como hex(), oct() y bin()

- **Convertir float a int**

Para convertir de float a int debemos usar float(). En este paso hay que tener precaución debido a que el número entero no permite almacenar decimales, por lo que lo que se encuentre después de la coma (,) se perderá.

```
a = 3.5
a = int(a)
print(a)
# Salida: 3
```

- **Convertir float a string**

Podemos convertir un float a string con str(). Podemos ver en el siguiente código como cambia el tipo de a después de hacer el *cast*.

```
a = 3.5
print(type(a)) # <class 'float'>
a = str(a)
print(type(a)) # <class 'str'>
```

- **Convertir string a float**

Podemos convertir un string a float usando float(). Es importante notar que se usa el. como separador.

```
a = "35.5"
print(float(a))
# Salida: 35.5
```

El siguiente código daría un error, dado que el punto no se reconoce por defecto como separador decimal.

```
a = "35,5"
print(float(a))
# Salida: ValueError: could not convert string to float: '35,5'
```

Y por último, resulta obvio pensar que el siguiente código dará un error también.

```
a = "Python"
print(float(a))
# Salida: ValueError: could not convert string to float: 'Python'
```

- **Convertir string a int**

Al igual que la conversión a float del caso anterior, podemos convertir de string a int usando int().

```
a = "3"
print(type(a)) # <class 'str'>
a = int(a)
print(type(a)) # <class 'int'>
```

Debemos tener cuidado, ya que no es posible convertir a int cualquier valor.

```
a = "Python"
a = int(a)
# ValueError: invalid literal for int() with base 10: 'Python'
```

- **Convertir int a string**

La conversión de **int a string** la podemos realizar con **str().**

A diferencia de otras conversiones, esta puede hacerse siempre, ya que cualquier valor entero que se nos ocurra poner en a, podrá ser convertido a **string**.

```
a = 10
print(type(a)) # <class 'int'>
a = str(a)
print(type(a)) # <class 'str'>
```

- **Convertir a list**

Es también posible hacer una conversión a lista, desde por ejemplo un set. Para ello podemos usar **list().**

```
a = {1, 2, 3}
b = list(a)

print(type(a)) # <class 'set'>
print(type(b)) # <class 'list'>
```

- **Convertir a set**

Y de manera completamente análoga, podemos convertir de lista a set haciendo uso de **set().**

```
x = ["Python", "Mola"]
y = set(a)

print(type(x)) # <class 'list'>
print(type(y)) # <class 'set'>
```

11. Comentarios

Los **comentarios de programación** son líneas de texto en el código fuente que el compilador ignora. Sabemos que el código fuente consta de instrucciones y los compiladores traducen las instrucciones del lenguaje de programación que usamos a lenguaje de máquina. Cuando el compilador encuentra un comentario, ignora esa línea o parte de ella y mira la siguiente declaración.

Los comentarios de programación se usan para aclarar el código, para que pueda comprender más fácilmente qué está haciendo el código, pero también se usan para reescribir una parte del código.

Con un sistema de control de versiones como GIT, no es necesario utilizar tanto los comentarios.

11.1 Ejemplos de comentarios

La mayoría de los lenguajes de programación usan la sintaxis del comentario en C, que es la siguiente:

```
/* Esto es un comentario */

Con la sintaxis /* */, podemos escribir en varias líneas

/* Esto

es

un

comentario */
```

En algunos lenguajes como en Java podemos usar los comentarios para crear documentación para tus clases o una variable o un método.

```
/** Comentario de documentación

* puede

* tener

* varias líneas

*/
```

C++ también añadió la opción de hacer un comentario en una línea:

```
//Comentario de tipo C++
```

En otros lenguajes, los comentarios se hacen de otra forma. Veamos algunos de ellos.

En Python utilizaremos # como símbolo para iniciar un comentario en línea.

También podemos escribir comentarios multilíneas, así:

```
#Comentario multilínea.

Segunda línea del comentario #
```

Comentario en HTML. En HTML, podemos escribir un comentario de esta manera:

```
<!-- Comentario HTML -->
```

Sabías que

Los comentarios en programación son líneas de texto que se utilizan para proporcionar explicaciones, aclaraciones o anotaciones dentro del código fuente de un programa. Estos comentarios no se ejecutan como parte del programa, sino que son ignorados por el compilador o intérprete.

La inclusión de comentarios en el código fuente tiene una larga historia y ha evolucionado junto con el desarrollo de los lenguajes de programación. Aquí hay un resumen de la historia de los comentarios en programación:

- **Lenguajes de programación tempranos:** En los primeros lenguajes de programación, como el Fortran y el Assembly, no se incluyeron características para agregar comentarios. Los programadores a menudo usaban convenciones de nomenclatura o escribían comentarios en secciones específicas del código para proporcionar explicaciones.
- **Algol 60:** Uno de los primeros lenguajes de programación en incluir comentarios fue Algol 60, desarrollado en la década de 1960. Algol 60 permitía el uso de comentarios de una sola línea utilizando el símbolo "%" al principio de la línea.

- **C y C++:** El lenguaje C, desarrollado en la década de 1970, introdujo los comentarios de estilo C utilizando los símbolos "//" para comentarios de una sola línea y "/* */" para comentarios de múltiples líneas. Esta convención se heredó en C++ y se ha convertido en una práctica común en muchos otros lenguajes.
- **Pascal:** El lenguaje Pascal, desarrollado a finales de la década de 1960, también incluía comentarios utilizando los símbolos "{" y "}" para comentarios de múltiples líneas y el símbolo "//" para comentarios de una sola línea.
- **Python, Ruby y otros lenguajes modernos:** Los lenguajes de programación más recientes, como Python y Ruby, también adoptaron la convención de comentarios de estilo C. Utilizan "//" para comentarios de una sola línea y "/* */" para comentarios de múltiples líneas. Además, en Python, se puede utilizar el símbolo "#" para comentarios de una sola línea.

En general, los comentarios son una práctica recomendada en la programación, ya que ayudan a otros programadores (y al propio programador en el futuro) a comprender el código y facilitan el mantenimiento y la depuración. Los comentarios bien escritos y descriptivos pueden mejorar la legibilidad y la claridad del código fuente. Sin embargo, es importante encontrar un equilibrio y no abusar de los comentarios excesivos, ya que un código limpio y autoexplicativo también es deseable.

12. Resumen

En esta unidad, se ha conocido los conceptos más relevantes de la programación, centrándonos en la estructura y bloques fundamentales, identificadores, palabras reservadas, variables, operadores y expresiones, así como la conversión de tipos.

La importancia de la estructura y los bloques son fundamentales en la programación. Aprendimos cómo organizar el código en bloques lógicos y cómo utilizar estructuras como bucles y condicionales para controlar el flujo de ejecución de un programa.

Los identificadores son nombres que le damos a elementos del programa, como variables, funciones o clases. Las palabras reservadas son términos que el lenguaje de programación utiliza para definir su sintaxis, y debemos evitar utilizarlos como identificadores. Las variables son contenedores que nos permiten almacenar y manipular datos durante la ejecución del programa.

Los operadores son símbolos especiales que nos permiten realizar operaciones matemáticas, lógicas y de comparación en nuestro código. Las expresiones, por otro lado, son combinaciones de variables, constantes y operadores que producen un resultado.

A veces, es necesario convertir un dato de un tipo a otro para poder operar con él o para adecuarlo al contexto de nuestro programa.

Glosario de términos

- **Constante:** En programación, una constante es un valor que se asigna una vez y no se modifica durante la ejecución del programa. Es un tipo de dato inmutable que se utiliza para representar valores fijos y predefinidos que se utilizan en el código.
- **Conversión:** La conversión en programación se refiere a la transformación de un tipo de dato a otro tipo de dato compatible. Puede ser necesario convertir datos cuando se necesita operar con ellos de diferentes maneras o cuando se requiere que cumplan ciertas restricciones o formatos.
- **Identificador:** En programación, un identificador se refiere a un nombre utilizado para identificar una entidad en el código, como variables, constantes, funciones, clases, etc. Los identificadores son palabras o combinaciones de letras, dígitos y caracteres especiales que siguen ciertas reglas establecidas por el lenguaje de programación utilizado.
- **Literal:** En programación, un literal se refiere a una notación o representación directa de un valor constante en el código fuente. Los literales son valores fijos que se escriben directamente en el código y no cambian durante la ejecución del programa.
- **Registro**: En programación, un registro (también conocido como estructura, struct o record en algunos lenguajes) es una forma de agrupar diferentes tipos de datos relacionados en una sola unidad. Un registro permite almacenar múltiples valores de diferentes tipos bajo un solo nombre y acceder a ellos de manera organizada. Un registro se define mediante una estructura de datos compuesta que contiene uno o más campos o miembros, cada uno con su propio nombre y tipo de datos. Los campos dentro de un registro pueden representar propiedades, características o atributos del objeto que se está modelando.
- **Precedencia:** se refiere al orden en el que se evalúan y resuelven las operaciones dentro de una expresión. Cada operador tiene una prioridad asignada, y aquellos con una mayor precedencia se evalúan antes que los de menor precedencia. Esto asegura que las operaciones se realicen de acuerdo a las reglas matemáticas y lógicas establecidas.

Tech Hub <FP> euroformac

Bibliografía

Programación orientada a objetos (2023) El Libro De Python. Available at: https://ellibrodepython.com/programacion-orientada-a-objetos.

El tutorial de python (2022) Python documentation. Available at: https://docs.python.org/es/3/tutorial/